

may proclaim the correct bound values associated with the excluded input in question, however, these values may be classically incorrect. Several of these scenarios exist, where the semantic mapping of real-valued bounds to classical states are erroneous. We highlight a few below.

For example, consider a binary conjunction with Unknown $[0, 1]$ and True $[1, 1]$ arguments, where the conjunction is itself False $[0, 0]$. Classicality demands that downward inference under this configuration should result in the Unknown input being set to False — since only a false and true input results in a false conjunction in the upward direction. The functional inverse, however, demands from the activation function configuration that a conjunction being exactly zero can only arise if all inputs were also exactly zero. All upper bound inputs are therefore pulled down to zero, irrespective of their current state. This incorrectly sets the True argument into a Contradictory state. A False Implication and True disjunction neuron behave similarly.

Also consider a False $[0, 1 - \alpha]$ binary conjunction with one pre-existing False $[0, 1 - \alpha]$ argument, A . Bi-direction inference demands that B remains the same by offering strictly Unknown $[0, 1]$ bounds — it should not be able to say anything about the source of falsity given an existing false input. The functional inverse may, however, aggressively tighten bounds beyond the base Unknown $[0, 1]$ state and further yet out of the classically Unknown $[1 - \alpha, \alpha]$ endpoint as alpha decreases.

To combat these concerns, downward inference can be seen as nothing but upward inference of a certain tautology. Since the tailored activation approach is classically correct, this automatically does the correct calculation for classical values. Beyond classical values, this can still be used but with a sanity check, namely the functional inverse.

To achieve this, the following tautology $B \rightarrow (A \rightarrow (A \& B))$ is conditionally applied, when trying to retrieve the value of B (the truth of A and $A \& B$ are assumed known and B takes the role of ‘excluded’ since we want to infer its value). The method we use ‘swaps’ out the implication and replaces it with equality, whose truth is equivalent to the truth of the converse $(A \rightarrow (A \& B)) \rightarrow B$:

Table 4: downward inference tautology

A	B	A&B	$A \rightarrow (A \& B)$	$B \rightarrow A \rightarrow (A \& B)$	$B = A \rightarrow (A \& B)$
0	0	0	1	1	0
0	1	0	1	1	1
1	0	0	0	1	1
1	1	1	1	1	1

If we have a look at the truth table 4, we note that equality does not hold in row 1. Despite the tautology being true for row 2, it is indistinguishable from row 1 and cannot be enforced (A and $A \& B$ are given, B is being calculated hence is treated as not given).

F.8.1 Learning from nothing

The tautology in table 4 has undesired non-classical behavior that is only brought to light when enforcing the non-classical characteristic of not ‘learning from nothing’. This test verifies that a Cartesian product of neuron inputs are incapable of retrieving tighter bounds from downward inference, given unknown neuron output bounds. It must be noted that a neuron may operate classically despite non-classical inputs, which further contributes to discrepancies.

By way of example, consider a classically False $[1 - \alpha, 1 - \alpha]$ conjunction with ($\alpha = 0.8$) and one True ($[\alpha, \alpha]$) argument, A . The functional inverse, not having any notion of classicality, offers $\sim$ False bounds $B = [0.4, 0.4]$, while the logical inference, correctly enforces classicality by offering False bounds $[0.2, 0.2]$. Notice that only a maximally True argument, $[1, 1]$, offers a False, $[0.2, 0.2]$, output for the functional inverse. Whereas the logical inference offers a $[0.1, 0.1]$ False bound. As the truth of the conjunction increases away from False $[1 - \alpha, 1 - \alpha]$ to $\sim$ Unknown $[0.5, 0.5]$, the functional inverse is correctly non-classical, however, the logical inference is undesirably not so — since the tautology first has to lift out of the classically-false range $[0.1, 0.2]$ before offering a non-classical value to B .

This behavior therefore motivates for a logical downward inference that is conditioned on the functional inverse outside of the classical regions. For the above scenario, the functional inverse updates too aggressively for both bounds in the classical regions. While in the non-classical region,

the lower bound is too aggressive but the upper bound is correct. Comparatively, the logical inference learns too much for the upper bound but just enough for the lower bound of the non-classical region. A combination of both approaches provide for more consistent bound behavior.

We therefore condition the logical inference tautology on the functional inverse by selecting the looser of the two bounds, while keeping the weight assignment consistent with (4) and (5):

$$U_{x_j} = \begin{cases} U_{x_j} & \text{if } U_{x_j} \geq \alpha \text{ or } U_{x_j} \geq f_{\mathbf{w}}^{-1}(\bigotimes_{i \neq j} L_{x_i}, U_{\bigotimes_i x_i}), \\ f_{\mathbf{w}}^{-1}(\bigotimes_{i \neq j} L_{x_i}, U_{\bigotimes_i x_i}) & \text{otherwise} \end{cases} \quad (98)$$

$$L_{x_j} = \begin{cases} L_{x_j} & \text{if } L_{x_j} \leq 1 - \alpha \text{ or } L_{x_j} \leq f_{\mathbf{w}}^{-1}(\bigotimes_{i \neq j} U_{x_i}, L_{\bigotimes_i x_i}), \\ f_{\mathbf{w}}^{-1}(\bigotimes_{i \neq j} U_{x_i}, L_{\bigotimes_i x_i}) & \text{otherwise} \end{cases} \quad (99)$$

To demonstrate this behavior of not ‘learning from nothing’, we test a binary conjunction ($A \& B$) with predefined input bounds to an **Unknown** neuron. The computation follows both a upward and a downward pass, offering new bounds to the neuron and thereafter offering argument updates.

The ground truths in Figure 6 (on the left) is characterised by an identity function for output argument B . The output $B = A \rightarrow (A \& B)$ (on the right) illustrates the effect of logical downward inference conditioned on equations (98) and (99), as α is varied. Ideally, a neuron should not ‘learn from nothing’ across all real-valued bounds, however learning inside the classical regions doesn’t impact the interpretation of the neuron and can be therefore be ignored — grayed out regions.

An additional caveat of the conditioned logical inference tautology is that contradictions also require conditioning. While the tautology may aggressively tighten bounds within a classical region, they still ensure a classical operation between regions. An intra-classical contradiction (e.g. where the lower bound is greater than the upper bound, but both bounds are greater than α or less than $1 - \alpha$) can therefore no longer be considered as such. Note that this does not break the interpretability of classical regions — while the bounds may disagree on the relative truth value, both agree on the same classical state. Lower and upper bounds within the ranges of $[0 \rightarrow 1 - \alpha, 0 \rightarrow \text{lower bound}]$ or $[\text{upper bound} \rightarrow 1, \alpha \rightarrow 1]$ are therefore no longer viewed as contradictions and the relevant loss terms are updated accordingly.

F.8.2 Modus ponens

It is worth noting that downward-inference is a generalization of classical modus ponens (or modus tollens). What is interesting with downward inference as implemented above is that downward inference is nothing but upward inference. This is surprising from a classical logic theory point of view, since modus ponens is a meta-rule, where as above, downward inference is recast as upward inference. It seems to indicate that the other machinery that we have included, namely, bounds, upward inference, aggregation, tailored-activations and so on, contain sufficient ‘meta’ structure to contain modus ponens naturally. This is a significant aesthetic vote-of-confidence for LNNs.

F.9 Learning

Weights are interpretably constrained by the tailored activation function and regularisation is no longer necessary. Losses from the predictive accuracy (supervised learning) and logical consistency checks (self-supervised learning) may be individually or simultaneously calculated.

It may often be the case that only a subset of root formulae or subformulae labels are available to learn from, if at all. Theorem proving is one such example, where grounded facts and formulae are assumed to be True. Self-supervised learning can then be used to propagate bounds from facts to formulae and decrease logical inconsistencies by propagating bounds downwards. These inference steps may be computed until bounds converge. As bounds are updated, automatic differentiation packages may be used to back-propagate gradients and update weights to correct logical inconsistencies. Weight updates may be determined by, but are not limited to, standard gradient descent methods.

After computing an inference step for a single neuron or across the entire network, parameters may be updated. The standard approach is to compute upward and downward inference until convergence, to accumulate gradients and thereafter do network-wide updates — where partial derivatives identify the source of losses. An alternative is to compute inference in both direction until convergence while

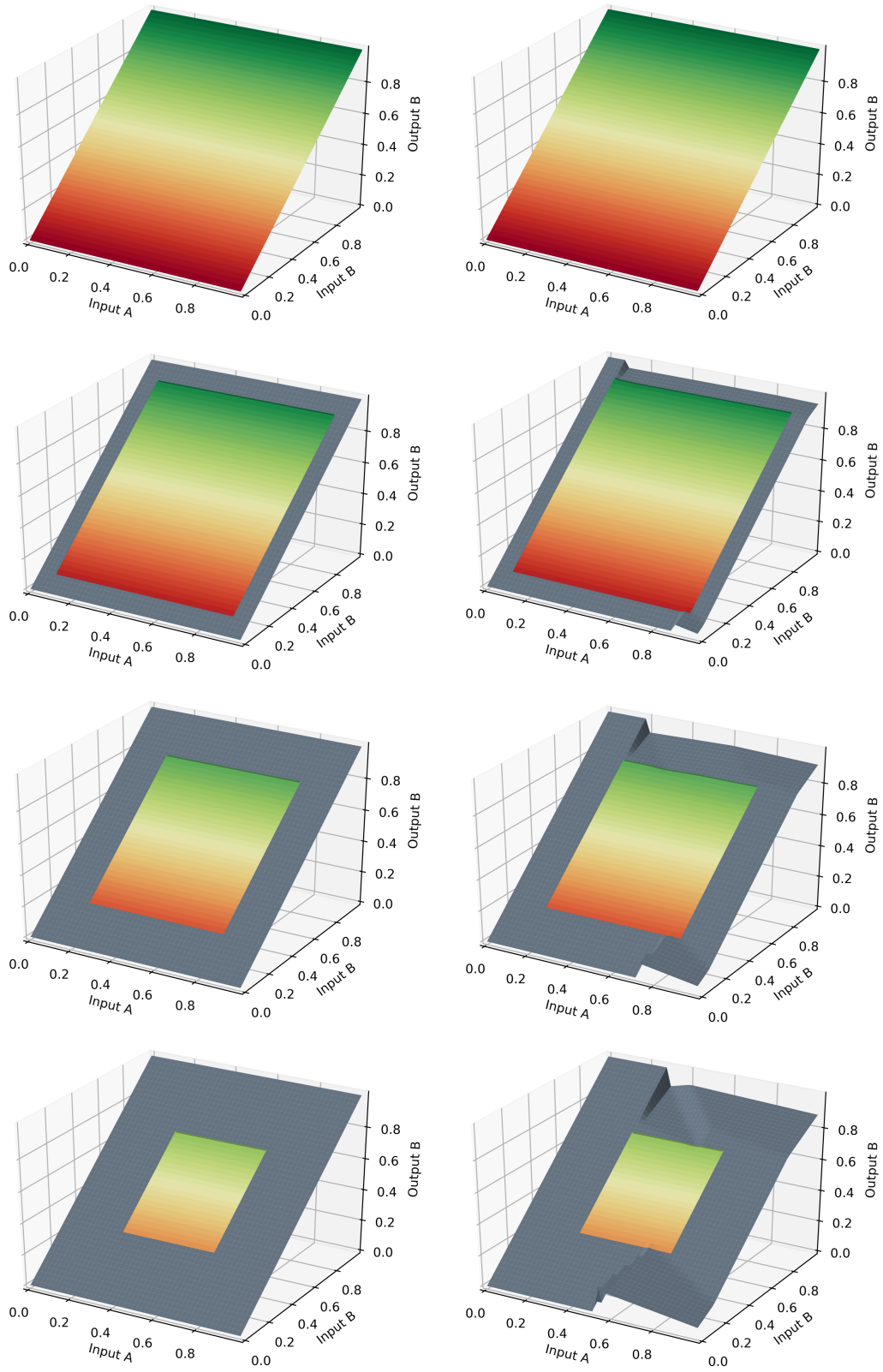


Figure 6: Testing that a neuron cannot learn from nothing; left: ground truth, right: output from binary conjunction argument, top-to-bottom: $\alpha = [1, 0.9, 0.8, 0.7]$